universität wien

MULTIOMICS DATA SCIENCE - FUNCTIONS & LIBRARIES

RECAP

- We stored information in variables and used / manipulated them
- With control structures, we can control the order of executing commands
- With collections, we can store several information of the same domain in one variable
- Different collections have differrent (dis)advantages

REUSING CODE

- To write efficient and lean code, we learned to reuse code
- Loops offer such a functionality, but are limited for a more generic usage, since the code has to be repeated immediately
- But we want to reuse certain code fragments at arbitrary positions

REUSING CODE

- To reuse code, there must be a mechanism with the following properties:
  - We need to jump from a certain position in our current execution to another certain position
  - The reused code should have the possibility to be dynamic, i.e. to compute something dependent of a certain input
  - We want that the reused code computes a value (or more) and stores / provides it for further usage
  - There must be a mechanism that after executing the reused code, we jump back to our previous position of where we launched the reused code

FUNCTIONS

- Functions are exactly such constructs (in Python and many other programming languages)
- Functions serve the purpose to capsulate code fragments and reuse it at any position
- Functions are already known in its basic form from mathematics
- For example: linear functions

BASIC IDEA OF FUNCTIONS

- Input: the function gets certain information
- Computation: the function is computing based on these information
- Output: the computed information is being returned

BASIC IDEA OF FUNCTIONS

- Input: the function gets certain information
- Computation: the function is computing based on these information
- Output: the computed information is being returned
- For example: $f(x) = 5x + 3$
  - Input: x value
  - Computation: linear transform
  - Output: y value

SYNTAX STRUCTURE OF FUNCTIONS

- A function contains the following syntax elements:
  - Keyword def
  - A function name, arbitrarily choosable (such as variable names)
  - Function brackets
  - Optional: arguments in the brackets, comma , separated
  - Colon
  - Function body
  - Optional: return command with a value

```
def my_function(argument1, argument2):
    command_1
    command_2
    return value
```

FOUR TYPES OF FUNCTIONS

- Functions without input / without output:

```
def my_function():
    ...
```

- Functions without input / with output:

```
def my_function():
    return value
```

- Functions with input / without output:

```
def my_function(argument1, ...):
    ...
```

- Functions with input / with output:

```
def my_function(argument1, ...):
    return value
```

universität wien

FIRST CORRECT FUNCTION

- Here is a full example for a function definition and call:

```
def addition(number_1, number_2):
    result = number_1 + number_2
    return result

res = addition(15, 3)
print(res)

# here is some other code in between

res_2 = addition(4, 5)
print(res_2)
```

FUNCTION CALLS

- As shown in the previous example, functions are called by their function name and round brackets ( ):

```
result = function_with_return(arg1)
function_without_return(arg)
```

- When we expect a return value from a function, it obviously has to be assigned to a variable
- The return value of a function without return command is `None`
- Functions without a return command don‘t need to be assigned to a variable

FUNCTION ARGUMENTS

- Functions don‘t need to be described with arguments, but arguments offer a bigger flexibility
- There are two types of arguments: mandatory and optional
- When calling a function, we need to provide as many mandatory arguments as defined in the function declaration (of course in the correct order)

OPTIONAL ARGUMENTS

- Optionals arguments can be, but don‘t need to be provided
- These arguments are marked within the function declaration by providing a default value:

```
def my_cool_function(arg1, arg2 = 0):
    ...

my_cool_function(val1)
my_cool_function(val1, val2)
```

- Caution: all mandatary arguments have to be declared first, the optional arguments are declared afterwards!

ASSIGN OPTIONAL ARGUMENTS

- Often functions possess many arguments with many optionals arguments:

```
def my_function(a, b, c = 0, d = 0, e = True, f = "foo"):
    ...
```

- It is often the case that we only need some of these optional arguments

ASSIGN OPTIONAL ARGUMENTS

- Often functions possess many arguments with many optionals arguments:

```
def my_function(a, b, c = 0, d = 0, e = True, f = "foo"):
    ...
```

- It is often the case that we only need some of these optional arguments

- We can achieve this by defining the argument assignment during the function call:

```
my_function(val1, val2, f = "bar")
```

RETURN OF SEVERAL VALUES

- A lovely feature of Python is that it natively supports the return of several values at once:

```
def sum_and_difference(number_1, number_2):
    the_sum = number_1 + number_2
    difference = number_1 - number_2
    return the_sum, difference

s, d = sum_and_difference(10, 4)
print(s, d)
```

- When calling such variables, we also need to assign as many variable left-hand side of the equation sign

LIBRARIES / MODULES

- Often we need standard calculations, e.g., average or standard deviation for a list of numbers
- Such calculations may by very complicated or their basic methods are not publically available
- A self implementation of such methods is prone to errors during coding
- Do we really have to invent the wheel over and over again?

LIBRARIES / MODULES

- Libraries are Python scripts, which offer functions, data structures, or algorithms on a narrow topic, e.g., statistical functions
- Python offers an easy possibility, to import such libraries into your own script
- In fact, libraries are nothing else but pre-written Python code
- Modules are libraries, that offer a big number of functions and algorithms on a complete domain, such as plotting

OWN LIBRARIES

- The implementation of own libraries has the advantage that often used code doesn‘t need to be copied into new scripts

- Here an example for a library, file name: mylib.py

```
def avg(numbers):
    return sum(numbers) / len(numbers)

def std(numbers):
    mue, std = avg(numbers), 0
    for num in numbers: std += (num - mue)**2
    return (std / len(numbers))**0.5
```

USE OWN LIBRARIES

- Library files need to be in the same folder as the executed script file in order to be imported

- With the `import` command, we can import the libraries

```
import mylib


mue = mylib.avg([1, 2, 3, 4, 5, 6])
```

- To avoid naming conflicts, function calls of libraries need to be provided with its origin (here `mylib`)

USE OWN LIBRARIES

- An alternative is to rename libraries during the import with the key word `as`:

```
import mylib as lib

mue = lib.avg([1, 2, 3, 4, 5, 6])
```

- Of course, we can freely set the name (here `lib`)

IMPORT FUNCTIONS DIRECTLY FROM LIBRARIES

- To import functions / variables directly from libraries, we can use the `from` command:

```
from mylib import avg, std
// to import everything from a lib:
// from mylib import *

mue = avg([1, 2, 3, 4, 5, 6])
```

- Now, we can directly call the `avg` function

MANY BUILTIN LIBRARIES

- Python offers many builtin libraries with different functions:

  - `math` different mathematical functions
  - `statistics` many statistical functions
  - `csv` basic routine to import csv files
  - `sys` library to access Python internal routines
  - `os` access on functions of the operating system

STATISTICS LIBRARY

- Functions from the `statistics` library are:

  - `mean` mean (average) value
  - `median` median
  - `geometric_mean` geometric mean
  - `pstdev` standard deviation of a population
  - `pvariance` variance of a population
  - `stdev` standard deviation of a sample
  - `variance` variance of a sample

```
from statistics import mean, pstdev
numbers = [1, 2, 3, 4, 5, 6]
mue, sigma = mean(numbers), pstdev(numbers)
```

MATH LIBRARY

- Functions from the `math` library are:

• `ceil`	integer roundup
• `fabs`	absolute value
• `factorial`	factorial
• `floor`	integer rounddown
• `exp`	natural exponential function
• `pow`	exponential function with arbitrary base
• `sqrt`	square root
• `log`	natural logarithm
• `sin, cos, ...`	All important trigonometric functions
• `pi, e`	mathematical constants

SCRIPT CALL WITH PARAMETERS

- Often scripts are written to be dependent on input parameters (best example: input file)

- Surely, one can hard code the file path in the code, however a cleaner solution is to call the Python script with input parameters (i.e., treat the Python script as a function itself):

```
> python3 myscript.py C:\path\to\table.xlsx
```

- In Spyder, you can provide input parameters via: Run > Configuration per file > Command line options

SCRIPT CALL WITH PARAMETERS

- To read the provided parameters from the script call, we must import the `sys` library

- All provided parameters are stored in the contained list attribute `argv`:

```
import sys
print(sys.argv)
// Output: ['myscript.py', 'C:\path\to\table.xlsx']
```

- Caution: the first argument in the list is always the name of the executed Python script

SPLIT FUNCTION AND FILE READING

- The `split` function expects a string and a pattern for splitting as input parameters and returns a list of the resulting tokens:

```
tokens = "do,not,split,here".split(",")
// ['do', 'not', 'split', 'here']
```

SPLIT FUNCTION AND FILE READING

- The `split` function expects a string and a pattern for splitting as input parameters and returns a list of the resulting tokens:

```
tokens = "do,not,split,here".split(",")
// ['do', 'not', 'split', 'here']
```

- With the `open` and `read` function, we can readin a complete file and split it into a list of lines in a single line of command:

```
my_data = open('C:\path\to\textfile.txt').read().split('\n')
```